

Multi-Product Inventory Optimization Agent

Date: 05/31/2026 | Instructor: Dr. Prateek Jaiswal

1. TEAM MEMBERS AND ROLES

Name	Email	Program	Primary component ownership
Prerna Jha	jha97@purdue.edu	MS BAIM	C1 — Problem Formulation Module
Kayal Gownder	kgownder@purdue.edu	MS BAIM	C2 — Data Collection & Preparation
Niteen Tomar	ntomar@purdue.edu	MS BAIM	C3 — Prediction Layer
Muhammad Shaharyar Amjad	amjad1@purdue.edu	MS BAIM	C4 — Optimization Engine
Tashfeen Khan	khan758@purdue.edu	MS BAIM	C5 — Explanation & Validation

2. BUSINESS DECISION PROBLEM AND CONTEXT

2.1 Business context

Retail inventory management is one of the most frequent, high-stakes decisions in operations. A category manager at a large-format grocery or general merchandise retailer must decide — before each selling period — how many units of each product to stock. This decision is made under demand uncertainty: actual customer demand will not be observed until after the order has been placed and inventory committed.

Current practice relies on one of two approaches: (a) a rolling-average heuristic, or (b) a prediction model that treats the mean forecast as the order quantity. Both share a critical flaw: they optimize prediction accuracy rather than decision quality. When overstocking and understocking carry asymmetric costs — as they always do in retail — ordering the mean forecast is systematically biased. The problem compounds across multiple SKUs: storage space and procurement budget are shared, so decisions cannot be made in isolation.

2.2 Problem class

This is a **multi-product stochastic inventory ordering problem**. At the start of each planning period (week), a retail buyer decides how many units of each SKU to order before demand is realized. The goal is to maximize expected profit across all SKUs — equivalently, to minimize expected overage costs (holding per unsold unit) plus underage costs (lost margin per unit of unmet demand) — subject to three binding constraints: (1) a per-SKU fill rate $\geq 92\%$, (2) a shared storage capacity limit, and (3) a total weekly procurement budget.

The critical insight from Lecture 2 (Bertsimas-Kallus) is that the optimal order quantity is **not** the mean demand forecast, but the critical ratio quantile: $Q^* = F^{-1}(c_u / (c_u + c_o))$. When $c_u > c_o$ — the typical retail case where lost sales cost more than holding — the agent must systematically order above the mean. No fixed rule or MSE-minimizing forecast achieves this automatically.

3. BACKWARD MAPPING

Element	Detail
Decision variables	Q_i : order quantity (units) per SKU i per planning period (week)

Objective	Maximize $\sum_i E[\text{profit}_i(Q_i)] = \sum_i E[p_i \cdot \min(D_i, Q_i) - c_i \cdot Q_i - h_i \cdot \max(Q_i - D_i, 0) - s_i \cdot \max(D_i - Q_i, 0)]$
Constraints	(1) $P(D_i \leq Q_i) \geq 0.92$ per SKU · (2) $\sum p_i Q_i \leq B = \$5,000/\text{cycle (est.)}$ · (3) $\sum Q_i \leq 1,200$ units (est.) · (4) $Q_i \geq 0$
Key parameters	Selling price p_i (M5 price table) · Unit cost $c_i = 0.70 \times p_i$ (30% gross margin) · Holding cost $h_i \approx 0.004 \times p_i/\text{week}$ (20% p.a.) · Stockout penalty $s_i = \text{lost margin per unit}$
Uncertain parameters	Weekly demand D_i per SKU — uncertain at decision time, predicted from historical M5 sales data
Data source	Walmart M5 Forecasting Competition (Kaggle) · Store CA_1 · HOUSEHOLD dept · 15 SKUs · Last 104 weeks

4. DATA PLAN

4.1 Dataset

Field	Details
Dataset	Walmart M5 Forecasting Competition — kaggle.com/competitions/m5-forecasting-accuracy
Size	3,049 products $\times$ 10 stores $\times$ 1,941 days (Jan 2011–Jun 2016). Includes sell_prices and calendar tables with SNAP and event flags.
Project scope	Store CA_1 · HOUSEHOLD department · 15 SKUs · last 104 weeks. Pipeline parameterized by store_id, dept_id, sku_list — rescopes without code changes.
Key features	Weekly unit demand (aggregated from daily) · avg_sell_price per SKU · SNAP flag · event type · week-of-year seasonality
Limitations	Sales $\neq$ true demand (stockout periods suppress observed demand). Missing prices imputed from nearest available week. Covers 2011–2016 only.

4.2 Train / test split

Chronological split (random splits would leak future demand into training).

- **Training set:** Weeks 1–78 (≈ 18 months). Distribution parameters and regression coefficients estimated on training data only.
- **Hold-out test set:** Weeks 79–104 (26 weeks). Agent performance evaluated using realized newsvendor cost vs. both baselines on actual demand.

5. AGENT ARCHITECTURE — FIVE REQUIRED COMPONENTS

5.1 Pipeline diagram



5.2 Component descriptions

C1 — Problem Formulation Module

Accepts a natural language or JSON problem description. A lightweight LLM layer (Claude API) parses free-text modifications and updates the relevant fields in the structured problem definition dict; output is schema-validated before any optimization runs. All downstream components read from this dict — no hard-coded problem-specific values anywhere. Handles new constraints, shifted objectives, changed cost structures, and different SKU sets via parameter update only.

C2 — Data Collection and Preparation

Loads M5 sales, price, and calendar files. Scopes to configured store/dept/SKU list. Aggregates daily sales to weekly demand. Merges SNAP flags and event indicators. Missing prices imputed from nearest available week (sensitivity-checked). Produces cleaned demand matrix and feature set for the prediction layer. EDA: distribution plots, time-series, zero-sale week rates per SKU.

C3 — Prediction Layer

Fits two candidate distributions per SKU to training demand: (a) Normal(μ , σ) via MLE, (b) Negative Binomial(r , p) via method of moments. KS test selects the better-fitting distribution per SKU. Reports fitted parameters, AIC comparison, and 90% bootstrap CI on mean demand. For SKUs with identifiable drivers, extends to a feature-adjusted mean: $\mu_{\text{adj}} = \mu_{\text{base}} + \beta_1 \cdot \text{SNAP} + \beta_2 \cdot \text{event}$ via Poisson regression. Outputs 1,000 Monte Carlo demand scenarios per SKU. Loss function: newsvendor cost, not MSE.

C4 — Optimization Engine

Formulation: Sample Average Approximation (SAA) of multi-SKU expected newsvendor cost with $N = 1,000$ demand scenarios. Solver: scipy SLSQP with fill rate, budget, and storage enforced as hard constraints.

Post-solve diagnostics: KKT verification (primal/dual feasibility, stationarity, complementary slackness). Shadow prices reported per binding constraint. Single-SKU validation against the newsvendor closed-form Q^* with $\leq 1\%$ discrepancy threshold before multi-SKU run.

Convexity: SAA objective is convex and piecewise linear in Q . All constraints are linear. SLSQP finds the provable global optimum — no multistart required.

C5 — Explanation and Validation Layer

Plain-language recommendation per SKU: order quantity, expected profit, fill rate achieved, expected leftover and stockout units. Sensitivity sweeps on holding cost, stockout penalty, fill rate target, and budget. Baseline comparison: (1) rolling 4-week average, (2) MSE-based predict-then-optimize. Benchmark problem runner with gap analysis vs. instructor solution.

6. BENCHMARK AND VALIDATION PLAN

6.1 Evaluating against baselines

Primary metric: Average expected newsvendor cost per SKU per week on the 26-week hold-out test set, computed identically for the agent and both baselines. A $\geq 10\%$ cost reduction vs. the rolling-average baseline is the practical significance threshold. The MSE-based predict-then-optimize baseline is expected to systematically underperform when $c_u > c_o$, directly illustrating the Lecture 2 prediction-to-prescription insight.

Secondary metrics: fill rate achieved per SKU, total holding vs. stockout cost decomposition, total expected profit on test set, shadow prices on binding constraints.

6.2 Analytical validation

On single-SKU subproblems, the SAA solution is compared against the closed-form $Q^* = F^{-1}(c_u / (c_u + c_o))$. A discrepancy $> 1\%$ fails the self-evaluation step and blocks the multi-SKU run. KKT conditions verified programmatically after every solver call.

6.3 Instructor benchmark

Agent run on the instructor-provided benchmark problem. Recommended Q^* and objective value compared to the instructor solution. **A gap of $< 5\%$ vs. the instructor solution is considered acceptable; all discrepancies will be analyzed** and explained — a gap is treated as a diagnostic opportunity, not a failure.

6.4 Stakeholder robustness

All problem parameters are configurable via the JSON problem definition dict output by C1 — parametric modifications require no code changes.

Modification type	Agent response
New constraint (e.g., per-SKU max, sustainability cap)	Add constraint dict to SLSQP; rerun; report new KKT shadow price
Changed cost structure	Update cost_params in JSON; optimizer reruns; critical ratio shifts automatically
Shifted objective	Update objective field; swap objective function; rerun; report new Q^*
Different SKU set or department	Update sku_list and dept_id; pipeline rescopes; prediction refits; optimizer reruns
New decision variable (structural change)	Requires formulation extension — estimated 1–2 hours; within two-day modification window

REFERENCES

- [1] Bertsimas, D. & Kallus, N. (2020). From Predictive to Prescriptive Analytics. *Management Science*, 66(3), 1025–1044.
- [2] Jaiswal, P. (2026). MGMT 590-037 Lecture 2: From Prediction to Prescription. Daniels School of Business, Purdue University.
- [3] Jaiswal, P. (2026). MGMT 590-037 Lecture 3: KKT Conditions. Daniels School of Business, Purdue University.
- [4] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition. *International Journal of Forecasting*, 38(4).
- [5] Zipkin, P. H. (2000). *Foundations of Inventory Management*. McGraw-Hill.